

Digital Banking System

Test Specification & OTP Verification Flow Verification Report

System: Digital Banking Microservices Platform	Date: September 24, 2026
Frontend: React 18 + Vite (Port 5173)	Gateway / Backend: Spring Cloud Gateway (:8080) / Transaction Service (:8082)
Architecture: SAGA Orchestration, Kafka, Redis, 2FA OTP	Test Status: All 7 Test Suites Passed

1. Executive Overview & Microservice Flow Pipeline

This specification verifies the complete React frontend flow supporting asynchronous **SAGA transaction orchestration**, **fraud detection heuristics**, **Redis OTP two-factor verification**, and **lifecycle polling**. Zero breaking changes were made to the backend services.

Transfer Category	Microservices Execution Flow	Terminal Status
Normal Transfer (Clean Activity)	Frontend → API Gateway (:8080) → Transaction Service (:8082) → Account Debit → Kafka (transaction.initiated) → Fraud Check (Clean) → Receiver Credit	COMPLETED
Suspicious Transfer (Velocity / 3x Avg / >90% Bal)	Frontend → Transaction Service (:8082) → Debit → Fraud Detection → Kafka (verification.required) → OTP stored in Redis (5 min TTL) → Status set to PENDING_VERIFICATION	PENDING_VERIFICATION
OTP Verification (User Submission)	Frontend POST /api/v1/transactions/{id}/verify?otp={otp} → Redis OTP matched → SAGA Completed → Kafka (transaction.completed)	COMPLETED
Wrong / Expired OTP (Security Guardrail)	POST /verify fails → SAGA Compensation: Sender balance credited back → Account blocked / Kafka (fraud.detected / transaction.refunded)	FLAGGED / REFUNDED

2. Detailed Test Scenarios & Verification Matrix

TEST 1: Normal Transfer (Clean Path — Non-Suspicious)	Status: VERIFIED / PASSED
Objective:	Verify execution of a standard transfer that does not trigger fraud rules.
Pre-conditions:	Sender account has sufficient balance (\$1,000.00). Destination account is active.
Execution Steps:	1. Open Transfer Money page. 2. Input destination account number and amount (\$10.00). 3. Click Authorize & Send Transfer (POST /api/v1/transactions/transfer).
Expected Result:	• Initial response returns status PROCESSING. • Frontend polls GET /api/v1/transactions/{id} at 2.5s intervals. • Fraud engine approves transaction; status transitions to COMPLETED. • Transfer Successful card displays with reference number, counterparty, and timestamp. • Balance and ledger statements refresh automatically.

TEST 2: Fraud Detection & OTP Verification Flow	Status: VERIFIED / PASSED
Objective:	Validate automatic OTP modal launch upon suspicious trigger and OTP verification.
Pre-conditions:	Sender account active. Fraud rules active (>3x average or >90% account balance).
Execution Steps:	1. Enter transfer amount exceeding fraud threshold (e.g., \$60.00). 2. Submit transfer request. 3. Polling detects status transition from PROCESSING to PENDING_VERIFICATION. 4. Modal opens; enter 6-digit OTP from Redis (verification:otp{id}). 5. Click Verify OTP (POST /api/v1/transactions/{id}/verify?otp={otp}).
Expected Result:	• Status automatically switches to PENDING_VERIFICATION. • OTP modal displays amount (\$60.00), shortened ID, 5-minute timer, and numeric input. • Upon valid submission, API returns COMPLETED. • Modal closes, success screen renders, and sender balance reflects deduction.
TEST 3: Invalid / Wrong OTP Handling & In-Place Retry	Status: VERIFIED / PASSED
Objective:	Ensure graceful degradation when an incorrect OTP is entered without closing modal.
Pre-conditions:	Transaction is in PENDING_VERIFICATION state with OTP modal active.
Execution Steps:	1. In the OTP modal, enter an incorrect 6-digit numeric code (e.g. 999999). 2. Click Verify OTP. 3. Observe API response and frontend UI reaction.
Expected Result:	• Verification fails gracefully without closing the modal. • Error banner displays: <i>'Invalid OTP. Please check the OTP and try again.'</i> • OTP input clears and allows user to re-enter the correct code.
TEST 4: Expired OTP & 5-Minute Timer Expiry Lifecycle	Status: VERIFIED / PASSED
Objective:	Verify system behavior when the 5-minute OTP lifespan lapses or Redis key expires.
Pre-conditions:	OTP modal open with active countdown timer (300 seconds).
Execution Steps:	1. Allow countdown timer to reach 00:00 or simulate Redis key TTL expiration. 2. Attempt OTP verification or inspect UI alert state.
Expected Result:	• Countdown timer counts down in real-time format mm:ss. • On expiry, frontend displays: <i>'OTP expired. Please request a new OTP.'</i> • SAGA compensation refunds the debited amount back to sender balance. • Action button provided to initiate a fresh transfer.
TEST 5: Asynchronous PROCESSING State & Polling Lifecycle	Status: VERIFIED / PASSED
Objective:	Validate polling mechanics, interval pacing, and unmount cleanup.
Pre-conditions:	User initiates transfer on frontend.
Execution Steps:	1. Submit transfer request. 2. Inspect Network tab for background GET /api/v1/transactions/{id} calls. 3. Navigate away or close modal while polling is active.
Expected Result:	• Polling triggers every 2.5 seconds up to safe threshold (25 polls / ~62s). • Processing card displays radar pulse animation and live counterparty information. • Component unmount cleanly halts all active setInterval instances without memory leaks.

TEST 6 & 7: Transaction Rejection & SAGA Refund (FAILED / FLAGGED / REFUNDED)	Status: VERIFIED / PASSED		
Objective:	Verify clear communication of system rejections and automated SAGA compensations.		
Pre-conditions:	User attempts transfer with invalid destination or triggers fraud block.		
Execution Steps:	1. Submit transfer to invalid destination or with blocked account status. 2. Review rendered outcome card and balance reflection.		
Expected Result:	• If FAILED: Displays clean error explanation without raw Java stack traces. • If FLAGGED/REFUNDED: Displays SAGA compensation banner, failure reason, amount refunded (+\$\$), and balance update.		

3. Frontend UX & Security Verification Checklist

Security / UX Requirement	Implementation Specification	Verification
6-Digit Numeric Enforcement	Input strictly sanitized with regex \D filter; non-numeric keystrokes prevented.	PASS
No Sensitive OTP Logging	OTP value is never passed to console.log or stored in localStorage.	PASS
Verification Button State	Button disabled until exact 6 digits are typed and disabled during in-flight API request.	PASS
Duplicate Request Prevention	Guarded by otpLoading state lock preventing concurrent submissions.	PASS
Friendly Error Shielding	Raw Spring/Kafka/Redis exceptions intercepted and mapped to user-friendly messages.	PASS
Live Balance Synchronization	Deductions and refunds immediately sync with global state and Navbar badge.	PASS

4. Verified API Endpoints Reference Table

HTTP Method & Path	Target Service	Payload / Parameters	Expected Status
POST /api/v1/transactions/transfer	Transaction Service (:8082)	{senderAccount, receiverAccount, amount, description}	201 CREATED (PROCESSING)
GET /api/v1/transactions/{id}	Transaction Service (:8082)	Path: transactionId	200 OK (COMPLETED / PENDING_VERIFICATION)
POST /api/v1/transactions/{id}/verify	Transaction Service (:8082)	Query Param: ?otp={otp}	200 OK (COMPLETED / FLAGGED)
GET /api/v1/accounts/{acc}/balance	Account Service (:8081)	Path: accountNumber	200 OK (BigDecimal balance)